Project Design Phase-II

Data Flow Diagram & User Stories

Date	09 March 2026
Team ID	698c67ecb31ff1dfca225748
Project Name	PlantCare AI: Advanced Plant Disease Detection Using Transfer Learning
Team Members	Akshat Bhati, Subhadipta Sarkar, Haridharshan B, Madan M
Maximum Marks	4 Marks

Data Flow Diagrams

A Data Flow Diagram (DFD) shows how data enters, flows through, and exits the PlantCare AI system. It shows what changes the information, where data is processed, and how results are delivered back to the user.

DFD Level 0 — Context Diagram (PlantCare AI)

The Level 0 (Context) DFD shows the system as a single process and its external entities.

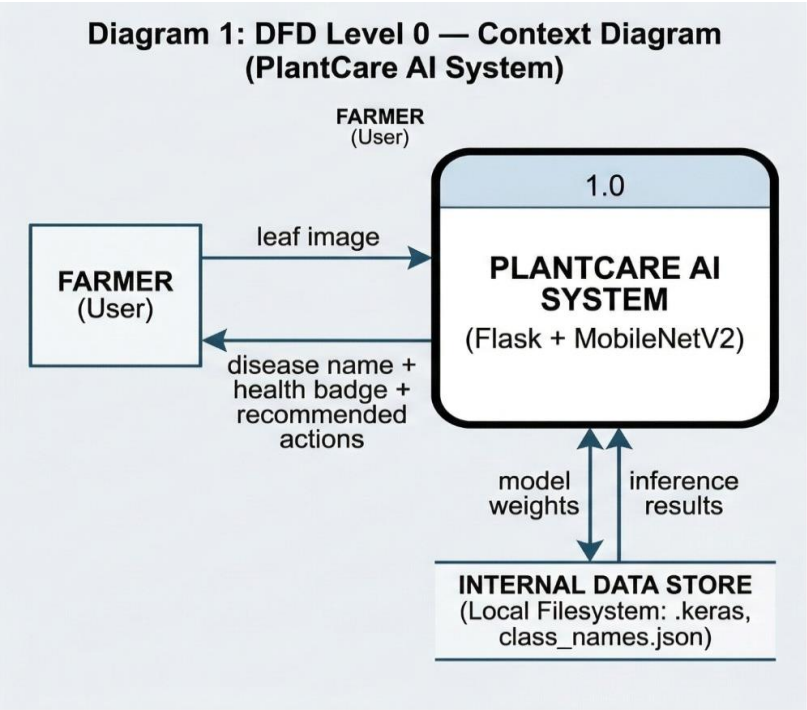
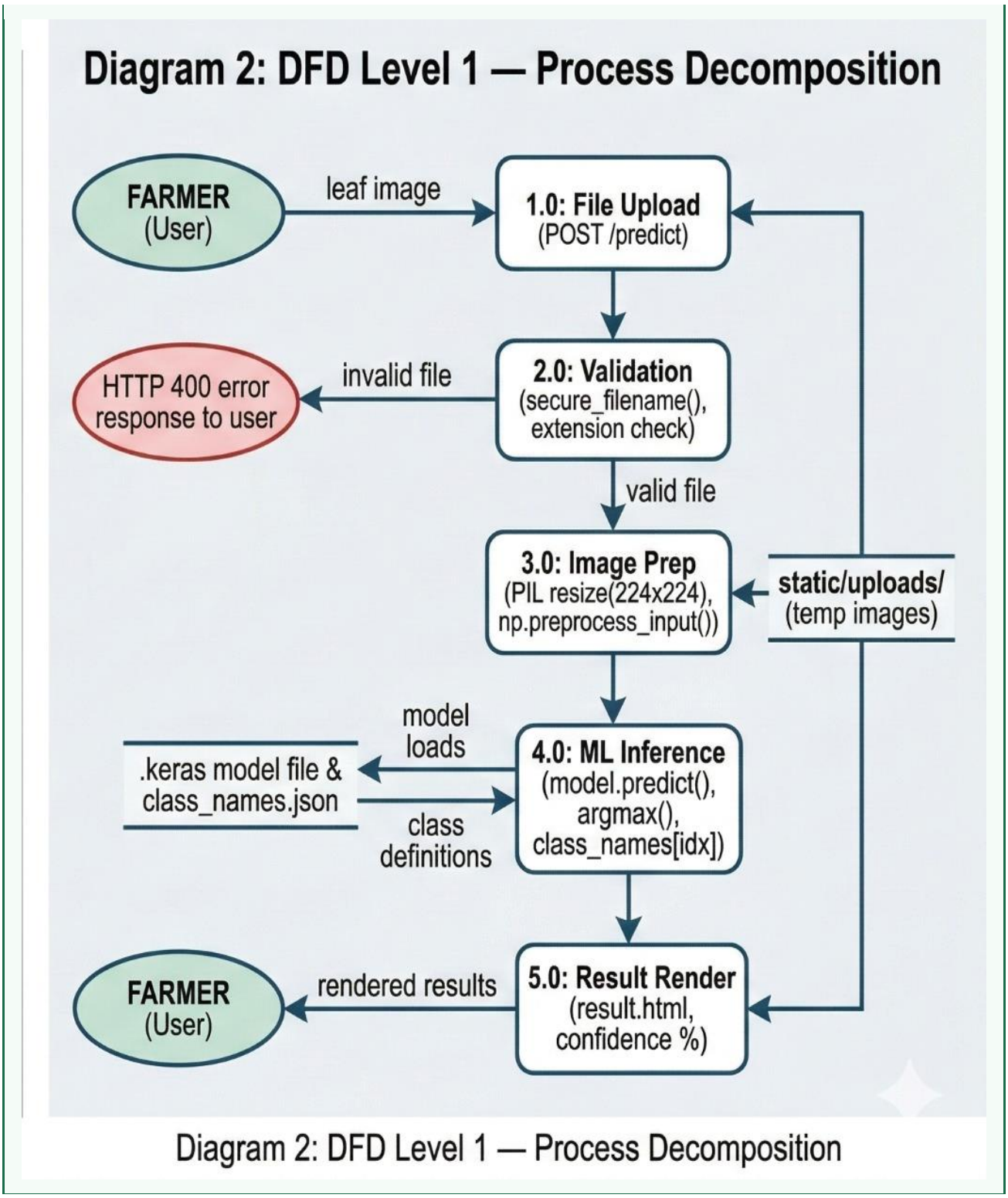


Diagram 1: DFD Level 0 — Context Diagram (PlantCare AI System)

DFD Level 1 — Process Decomposition

The Level 1 DFD breaks down the PlantCare AI system into its 5 main processes and shows data flows between them.



DFD Level 1 — Data Flow Description

#	Process	Input Data	Transformation	Output Data
1	File Upload (POST /predict)	Multipart HTTP request containing raw image file from browser	Werkzeug saves file; secure_filename() sanitises filename; validates extension (.jpg/.jpeg/.png/.gif)	Sanitised image file saved to static/uploads/[filename]; filepath stored in memory
2	Image Preprocessing	Raw image file at static/uploads/ filepath	PIL.Image.open() → convert('RGB') → resize(224,224) → np.array() → expand_dims(axis=0) → preprocess_input() normalisation	NumPy array of shape (1,224,224,3) with MobileNetV2-normalised pixel values, ready for inference
3	ML Inference	NumPy array (1,224,224,3) + loaded Keras model	model.predict() runs forward pass through MobileNetV2; np.argmax() selects highest probability index	Integer class index (0–37); confidence float (0.0–1.0)
4	Label Mapping	Integer class index + class_names.json list	class_names[index] retrieves string label; label parsed to extract plant name and disease name	Human-readable disease label string (e.g., 'Tomato___Late_blight')
5	Session & Redirect	Prediction label + image URL + confidence	Flask session stores prediction data; HTTP 302 redirect to GET /result	Encrypted session cookie written to client; user redirected to result page
6	Result Rendering	Flask session data + Jinja2 result.html template	Jinja2 template receives disease name, plant name, health badge logic, image URL, recommendations	Fully rendered HTML result page displayed to user

User Stories

The table below lists all user stories for PlantCare AI across all user types (Farmer, Developer, Administrator) and epics, following the standard Agile user story format with acceptance criteria, priority, and sprint release.

User Type	Functional Requirement (Epic)	USN	User Story / Task	Acceptance Criteria	Priority	Release
Farmer (Web / Mobile)	Image Upload	**USN-1**	As a farmer, I can upload a plant leaf photo by clicking 'Choose Image' on the	Upload page shows image preview; file saved	**High**	Sprint-5

User Type	Functional Requirement (Epic)	USN	User Story / Task	Acceptance Criteria	Priority	Release
			upload page and selecting a file from my phone	to server; form submits successfully		
Farmer (Web / Mobile)	Image Upload	**USN-2**	As a farmer, I can upload a plant leaf image by dragging and dropping it onto the upload zone on the upload page	Dropped file appears as preview in upload zone; file type accepted; form ready for submission	**High**	Sprint-5
Farmer (Web / Mobile)	Image Upload	**USN-3**	As a farmer, I see a real-time image preview of my selected file before I click the Analyse button	Selected image renders in the preview box within 1 second of selection; preview is accurate	**High**	Sprint-5
Farmer (Web / Mobile)	Image Upload	**USN-4**	As a farmer, I am shown a clear error message if I try to submit without selecting any image file	Page displays 'Please select an image file' error message; no server request is made	**High**	Sprint-5
Farmer (Web / Mobile)	Disease Classification	**USN-5**	As a farmer, I want my uploaded leaf image classified into one of 38 plant disease categories within 3 seconds of submission	Server returns prediction within 3 seconds; result page loads correctly with prediction data	**High**	Sprint-5
Farmer (Web / Mobile)	Disease Classification	**USN-6**	As a farmer, I want the system to detect whether my plant is healthy or diseased and display an appropriate colour-coded badge	RED badge for diseased plants; GREEN badge for healthy plants; badge text matches classification	**High**	Sprint-5
Farmer (Web / Mobile)	Result Display	**USN-7**	As a farmer, I can see my uploaded leaf image displayed on the result page alongside the AI diagnosis	Uploaded image renders correctly at appropriate size next to the diagnosis panel	**High**	Sprint-5
Farmer (Web / Mobile)	Result Display	**USN-8**	As a farmer, I can see the disease name and plant type clearly displayed on the result page in simple, readable text	Plant name and disease name parsed from class label and displayed in separate fields	**High**	Sprint-5
Farmer (Web / Mobile)	Result Display	**USN-9**	As a farmer, I can read specific recommended treatment or care actions for the disease identified on my plant	Result page displays at least 3 disease-specific treatment recommendations or care instructions	**High**	Sprint-5
Farmer (Web / Mobile)	Re-Analysis	**USN-10**	As a farmer, I can click 'Analyse Another Image' on the result page to return to the upload page and run a fresh diagnosis	Button navigates to /upload; session cleared; new upload form is empty and ready	**Medium**	Sprint-5
Farmer (Web / Mobile)	Navigation	**USN-11**	As a farmer, I can access the Home page and see a hero section with a 'Diagnose Your Plant' call-to-action button	Home page loads at GET /; hero section visible; CTA button links to /upload page	**High**	Sprint-5
Farmer (Web / Mobile)	Navigation	**USN-12**	As a farmer, I can read the About page to understand how PlantCare AI works,	About page loads at GET /about; 3-step How It Works flow visible;	**Medium**	Sprint-5

User Type	Functional Requirement (Epic)	USN	User Story / Task	Acceptance Criteria	Priority	Release
			what the model does, and its accuracy	accuracy stat (96.6%) displayed		
Developer	Data Collection (Epic 1)	USN-13	As a developer, I can set up Google Colab, install Kaggle API, and download the New Plant Diseases Dataset (87K+ images, 38 classes)	Dataset downloaded; train path and valid path confirmed; class count = 38	High	Sprint-1
Developer	Data Collection (Epic 1)	USN-14	As a developer, I can configure ImageDataGenerator with augmentation (flip, rotate, zoom, shift) for training and preprocessing-only for validation	Train generator: 70,295 images loaded; valid generator: 17,572 images loaded; class_indices confirmed	High	Sprint-1
Developer	Model Building (Epic 2)	USN-15	As a developer, I can load MobileNetV2 (include_top=False, imagenet weights) and add a custom classification head	model.summary() shows correct architecture; total params ~4.2M; output shape (None,38)	High	Sprint-2
Developer	Model Building (Epic 2)	USN-16	As a developer, I can freeze all base layers then unfreeze the last 10 layers for domain-specific fine-tuning	model.trainable_weights count shows ~732K trainable params; last 10 layers set trainable=True	High	Sprint-2
Developer	Model Training (Epic 3)	USN-17	As a developer, I can train the model with ModelCheckpoint, ReduceLROnPlateau, and EarlyStopping callbacks over 5 epochs	Training completes; best model saved as .keras; val_accuracy above 94% achieved	High	Sprint-3
Developer	Model Training (Epic 3)	USN-18	As a developer, I can export the best .keras model and class_names.json from Colab to local machine after training completes	Both files downloadable from Colab; model loads without error in Flask app environment	High	Sprint-3
Developer	Evaluation (Epic 4)	USN-19	As a developer, I can plot training vs validation accuracy and loss curves to verify model convergence and absence of overfitting	Accuracy curves converge; val_accuracy >= 96%; train-val gap < 3%; loss curves decrease monotonically	High	Sprint-4
Developer	Flask App (Epic 5)	USN-20	As a developer, I can build app.py with 5 Flask routes (GET /, /about, /upload, POST /predict, GET /result) and deploy locally	All 5 routes return correct HTTP 200 status; sessions work; prediction result correct for test images	High	Sprint-5
Developer	Flask App (Epic 5)	USN-21	As a developer, the predict_image() function correctly preprocesses any uploaded leaf image and returns the right disease class name	Function tested on 5 sample images; returns correct class label matching expected disease; inference < 2s	High	Sprint-5
Admin	System Maintenance	USN-22	As an administrator, I can replace the .keras model file with a retrained version	New .keras file dropped in app directory; Flask restart loads new model;	Medium	Sprint-5

User Type	Functional Requirement (Epic)	USN	User Story / Task	Acceptance Criteria	Priority	Release
			without changing any Flask application code	predictions use updated weights		
Admin	System Maintenance	**USN-23**	As an administrator, I can add or modify class names in class_names.json to update disease labels without modifying Python code	Updated class_names.json loaded at next Flask restart; new labels appear correctly in result page	**Low**	Sprint-5